Hogwarts Middle School Performance and Records System

Rebecca Gautreaux (C00460817) and Reem Tammam (C00563548)

CMPS 460-001: Database Management Systems

Project Requirements

Description:

This database was constructed to manage information about the teachers, students, and classes of Hogwarts Middle School. It tracks multiple students, their professors, and the classes offered through the school. Each student is associated with a grade level, has a GPA, and may have disciplinary infractions recorded. Students may also have preferences such as favorite subjects and teachers. Teachers are responsible for one or more classes and are described by their years of teaching experience, the subjects and grade levels they cover, and their salary.

The primary three relations are Students, Teachers, and Classes. The Students relation contains each student's name, age, gender, grade, GPA, favorite subject, favorite teacher, and number of disciplinary infractions. The teacher's relation stores each teacher's name, age, gender, subject taught, grade taught, years of teaching, number of students, class grade averages, and salary. The Classes relation tracks each subject offered, the grade it's taught to, the professor who teaches it, the course difficulty, and the course supply fee. However, since our proposal, we have added several more tables to connect students, classes, and teachers. Enrollment tracks which students are enrolled in which classes, and provides a basis for managing rosters and class performance. Discipline_log stores all data regarding each student's recorded discipline issues. Absences records which students are absent from which classes. Users track which teachers have access to the database, their usernames, and their passwords. With this database, the school administration will be able to query, update, and analyze information to improve academic management.

When making our proposal, we wrote several queries for our database to answer:

A) Retrieve the number of students whose preferred subject is English or Math.

```
MariaDB [hogwarts_middle_school_performance_and_records_system]> SELECT COUNT(StudID) AS Stud_Count FROM students WHERE FavSubject = 'Mathematics' OR FavSub
ject = 'English Literature';
+-------------+
| Stud_Count |
+-------------+
|           3 |
+-------------+
1 row in set (0.001 sec)
```

B) List the names of male students in ascending order based on age.

```
MariaDB [hogwarts_middle_school_performance_and_records_system]> SELECT FName, LName, Age FROM students ORDER BY age ASC;
+----------+--------+-----+
| FName    | LName  | Age |
+----------+--------+-----+
| Emma     | Stone  |  12 |
| Mason    | Hayes  |  12 |
| Noah     | Diaz   |  12 |
| Isabella | Foster |  13 |
| Lucas    | Morris |  13 |
| Sophia   | Green  |  13 |
| Liam     | Brooks |  13 |
| Ava      | Carter |  14 |
| Mia      | Parker |  14 |
| Ethan    | Reed   |  14 |
+----------+--------+-----+
10 rows in set (0.000 sec)
```

C) Find the average class size of female teachers.

D) Identify how many teachers have taught for over five years and do not teach Science.

```
MariaDB [hogwarts_middle_school_performance_and_records_system]> SELECT ProfID, FName, LName FROM professors WHERE Subject <> 'Science' AND YearTeach > 5;
+--------+---------+----------+
| ProfID | FName   | LName    |
+--------+---------+----------+
|    110 | Olivia  | Bennett  |
|    112 | Sophia  | Lopez    |
|    113 | Liam    | Hughes   |
|    114 | Isabella | Nguyen  |
|    115 | Jackson | Perez    |
|    117 | Lucas   | Morgan   |
|    118 | Mia     | Patel    |
|    119 | Noah    | Anderson |
+--------+---------+----------+
8 rows in set (0.000 sec)
```

E) Update class rosters by reassigning students based on GPA and disciplinary infractions.

F) Report how many male students are enrolled in a particular teacher's class.

G) List the names and ages of teachers with salaries above $69,554 who teach 7th grade.

```
MariaDB [hogwarts_middle_school_performance_and_records_system]> SELECT ProfID, FName, LName FROM professors WHERE Grade = 7 AND YearlySalary > 69554;
+--------+-------+-------+
| ProfID | FName | LName |
+--------+-------+-------+
|    118 | Mia   | Patel |
+--------+-------+-------+
1 row in set (0.000 sec)
```

<u>Data Modelling</u>

LOW-LEVEL MODELLING

ABSENCES(**<u>AbsenceID [PK]</u>**, ***StudID [FK]***, ***ClassID [FK]***, Date, Reason)

CLASS(**<u>ClassID [PK]</u>**, Subject, Grade, ***ProfID [FK]***, Difficulty, SupplyFee)

DISCIPLINE_LOG(**<u>LogID [PK]</u>**, ***StudID [FK]***, Date, Description, Severity)

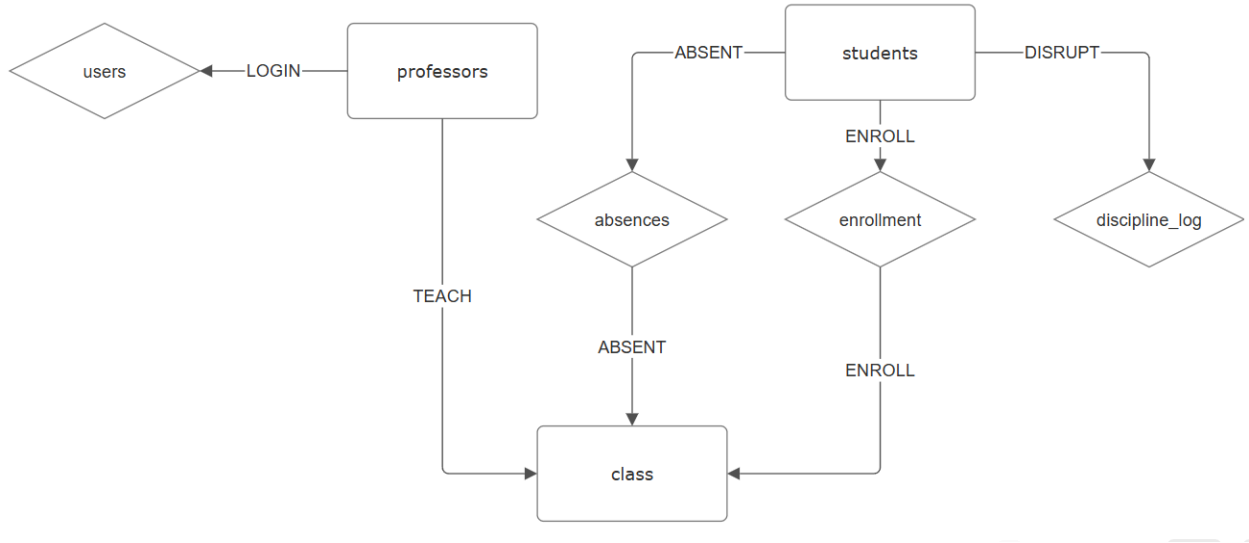
ENROLLMENT(**<u>EnrollmentID [PK]</u>**, ***StudID [FK]***, ***ClassID [FK]***, FinalGrade, Attendance)

PROFESSORS(**<u>ProfID [PK]</u>**, FName, LName, Age, Gender, Subject, Grade, YearTeach, Yearly Salary)

STUDENTS(**<u>StudID [PK]</u>**, FName, LName, Age, Gender, Grade, GPA, ClassCount, FavSubject, FavTeacher, DisciplineProbs)

USERS(**<u>UserID [PK]</u>**, Username, PasswordHash, Role, ***ProfID [FK]***)

HIGH-LEVEL MODELLING (ER DIAGRAM)



SIMPLIFIED RELATIONSHIP CHART

Table	Connected To	Why
students	enrollment, discipline_log, absences	Tracks all student-related events
professors	class, users	Teachers teach classes + have logins
class	enrollment, absences	Students enroll and may be absent

enrollment	students + class	Which student takes which class
discipline_log	students	Behavior tracking
absences	students + class	Attendance tracking
users	professors	Login system

This proves our database meets the requirements to be considered **3NF**. For relationships between two tables involving multiple primary keys, new tables, such as **enrollment** and **absences,** were created to meet the requirements of **3NF**.

Implementation

Table-by-Table Explanation and Purpose

Below is a clear explanation of **every table**, why it exists, and what role it plays in the Hogwarts Middle School Performance & Records System.

---

1. students Table

Purpose:

Stores all core information about each student at the school.

Why it exists:

Every other student-related table references this one — it is the central student database.

What it contains:

- Full name
- Age (with validation: 10–18)
- Gender
- Grade level (1–8)
- GPA
- Number of classes they take
- Favorite subject/teacher
- Discipline problem count

How it is used:

- Enrollment pages look up student IDs
- Discipline logs reference students
- Absence records reference students
- Reports (GPA, attendance, discipline, performance) pull from here

This is the foundation of all student-related data.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**StudID**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
2	**FName**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
3	**LName**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
4	**Age**	int(11)			No	*None*			Change Drop More
5	**Gender**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
6	**Grade**	int(11)			No	*None*			Change Drop More
7	**GPA**	decimal(3,2)			No	*None*			Change Drop More
8	**ClassCount**	int(11)			No	0			Change Drop More
9	**FavSubject**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
10	**FavTeacher**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
11	**DisciplineProbs**	tinyint(4)			No	0			Change Drop More

StudID	FName	LName	Age	Gender	Grade	GPA	ClassCount	FavSubject	FavTeacher	DisciplineProbs
1000	Emma	Stone	12	Female	6	3.50	5	Mathematics	Bennett	1
1001	Liam	Brooks	13	Male	7	3.70	6	History	Lopez	0
1002	Ava	Carter	14	Female	8	3.90	7	English Literature	Nguyen	0
1003	Noah	Diaz	12	Male	6	3.20	5	Science	Cole	2
1004	Sophia	Green	13	Female	7	3.80	6	Biology	Patel	0
1005	Ethan	Reed	14	Male	8	3.40	7	Computer Science	Morgan	1
1006	Isabella	Foster	13	Female	7	3.60	6	Art	Reed	0
1007	Mason	Hayes	12	Male	6	3.10	5	Physical Education	Hughes	3
1008	Mia	Parker	14	Female	8	4.00	7	English Literature	Nguyen	0
1009	Lucas	Morris	13	Male	7	3.30	6	Geography	Perez	1

---

2. professors Table

Purpose:

Stores teacher information for all subjects and grade levels.

Why it exists:

Each class needs to be linked to the teacher who teaches it.

What it contains:

- Name, age, gender
- Subject taught

- Grade level taught
- Years of teaching experience
- Salary

How it is used:

- The class table references ProfID
- The users table links logins to professor accounts
- Reports can show:
    - teacher performance
    - class rosters
    - salaries and staffing

This table essentially represents the staff directory.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**ProfID**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
2	**FName**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
3	**LName**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
4	**Age**	int(11)			No	*None*			Change Drop More
5	**Gender**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
6	**Subject**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
7	**Grade**	int(11)			No	*None*			Change Drop More
8	**YearTeach**	int(11)			No	*None*			Change Drop More
9	**YearlySalary**	int(11)			No	*None*			Change Drop More

ProfID	FName	LName	Age	Gender	Subject	Grade	YearTeach	YearlySalary
110	Olivia	Bennett	39	Female	Mathematics	6	12	65000
111	Ethan	Cole	42	Male	Science	8	15	72000
112	Sophia	Lopez	35	Female	History	7	9	61000
113	Liam	Hughes	50	Male	Physical Education	6	22	58000
114	Isabella	Nguyen	44	Female	English Literature	8	17	70000
115	Jackson	Perez	38	Male	Geography	7	11	64000
116	Ava	Reed	29	Female	Art	6	5	53000
117	Lucas	Morgan	47	Male	Computer Science	8	20	76000
118	Mia	Patel	41	Female	Biology	7	14	71000
119	Noah	Anderson	33	Male	Music	6	8	55000

---

3. class Table

Purpose:

Stores information about every class offered at the school.

Why it exists:

Students enroll in classes, and professors teach them — this table connects those two worlds.

What it contains:

- Subject name
- Grade level

- Teacher who teaches the class (ProfID)
- Difficulty rating
- Supply fee

How it is used:

- Enrollment assigns students to specific classes
- Absences link to a class
- Teachers view their classes via the login system

It defines what courses exist in the school.

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**ClassID**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
☐	2	**Subject**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
☐	3	**Grade**	int(11)			No	*None*			Change Drop More
☐	4	**ProfID**	int(11)			No	*None*			Change Drop More
☐	5	**Difficulty**	tinyint(4)			No	*None*			Change Drop More
☐	6	**SupplyFee**	int(11)			No	*None*			Change Drop More

ClassID	Subject	Grade	ProfID	Difficulty	SupplyFee
1	Mathematics	6	110	8	40
2	Science	8	111	7	50
3	History	7	112	6	30
4	Physical Education	6	113	5	20
5	English Literature	8	114	9	45
6	Geography	7	115	7	35
7	Art	6	116	4	25
8	Computer Science	8	117	9	60
9	Biology	7	118	8	55
10	Music	6	119	5	30

---

4. enrollment Table

Purpose:

Represents which students are enrolled in which classes.

Why it exists:

This table is the heart of the academic system — it connects *students* → *classes*.

What it contains:

- Student ID
- Class ID
- Final grade for that class
- Attendance percentage

How it is used:

- Shows each student's schedule
- Generates report cards
- Calculates GPA, average grades
- Lets teachers see who is in their classes

This is a junction table (many-to-many relationship) between students and classes.

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action		
☐	1	**EnrollmentID**	int(11)			No	*None*		AUTO_INCREMENT	Change	Drop	More
☐	2	**StudID**	int(11)			No	*None*			Change	Drop	More
☐	3	**ClassID**	int(11)			No	*None*			Change	Drop	More
☐	4	**FinalGrade**	decimal(4,2)			Yes	*NULL*			Change	Drop	More
☐	5	**Attendance**	int(11)			Yes	*NULL*			Change	Drop	More

EnrollmentID	StudID	ClassID	FinalGrade	Attendance
1	1000	1	95.00	98
2	1000	3	92.00	97
3	1000	5	96.00	99
4	1000	7	94.00	96
5	1000	9	93.00	95
6	1001	2	91.00	96
7	1001	4	89.00	94
8	1001	6	93.00	98
9	1001	8	95.00	97
10	1001	10	90.00	95
11	1002	1	98.00	100
12	1002	2	96.00	99
13	1002	3	97.00	100
14	1002	4	95.00	98
15	1002	5	99.00	100
16	1003	6	85.00	90
17	1003	7	87.00	92
18	1003	8	88.00	93
19	1003	9	86.00	91
20	1003	10	89.00	94
21	1004	1	94.00	98
22	1004	4	93.00	96
23	1004	6	95.00	97
24	1004	8	97.00	99
25	1004	10	92.00	95
26	1005	2	88.00	94
27	1005	3	90.00	96
28	1005	5	92.00	97
29	1005	7	91.00	95
30	1005	9	89.00	93
31	1006	1	93.00	97
32	1006	3	91.00	95
33	1006	5	92.00	96
34	1006	7	94.00	98
35	1006	9	90.00	94
36	1007	4	82.00	88

37	1007	5	84.00	90
38	1007	6	83.00	89
39	1007	7	85.00	91
40	1007	8	86.00	92
41	1008	2	97.00	99
42	1008	4	98.00	100
43	1008	6	96.00	98
44	1008	8	99.00	100
45	1008	10	97.00	99
46	1009	1	88.00	94
47	1009	2	87.00	93
48	1009	3	89.00	95
49	1009	4	90.00	96
50	1009	5	91.00	97

---

5. discipline_log Table

Purpose:

Stores individual discipline incidents for students.

Why it exists:

A student may have 0, 1, or many discipline events — this table tracks each one.

What it contains:

- Which student was disciplined
- Date of incident
- Description

- Severity (Low/Medium/High)

How it is used:

- Discipline reports
- Behavior tracking
- Teacher dashboards
- Admin-level summaries

This supports behavioral monitoring and reporting.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**LogID**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
2	**StudID**	int(11)			No	*None*			Change Drop More
3	**Date**	date			No	curdate()			Change Drop More
4	**Description**	text	utf8mb4_general_ci		No	*None*			Change Drop More
5	**Severity**	enum('Low', 'Medium', 'High')	utf8mb4_general_ci		No	Low			Change Drop More

LogID	StudID	Date	Description	Severity
1	1000	2025-10-12	Minor disruption during class	Low
2	1003	2025-10-01	Talking during instruction time	Low
3	1003	2025-10-15	Failure to follow safety rules	Medium
4	1007	2025-09-20	Horseplay during PE	Low
5	1007	2025-09-28	Ignored teacher instructions	Medium
6	1007	2025-10-03	Pushed another student	High
7	1009	2025-10-22	Late to class repeatedly	Low

---

6. absences Table

Purpose:

Tracks only the days when a student is absent from a specific class.

Why it exists:

Instead of storing attendance for all days (which is huge), we only store absence days.

What it contains:

- Student ID
- Class ID
- Date of absence
- Reason (optional)

How it is used:

- Teacher portals to monitor student attendance
- Reports showing frequent absences
- Warning flags for students with patterns of missing class

This keeps attendance lightweight and efficient.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**AbsenceID**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
2	**StudID**	int(11)			No	*None*			Change Drop More
ance_and_records_system		nt(11)			No	*None*			Change Drop More
4	**Date**	date			No	*None*			Change Drop More
5	**Reason**	varchar(255)	utf8mb4_general_ci		Yes	*NULL*			Change Drop More

AbsenceID	StudID	ClassID	Date	Reason
1	1003	6	2025-11-02	Sick
2	1007	4	2025-11-02	Family emergency
3	1007	7	2025-11-10	Overslept
4	1009	1	2025-11-05	Doctor appointment

---

7. users Table

Purpose:

Stores accounts for logging into the web application.

Why it exists:

To implement security (required by project instructions).

What it contains:

- Username
- Hashed password (will generate actual hashed passwords using php)
- Role (admin or teacher)
- Optional professor link (teacher accounts connect to professors)

How it is used:

- Login page

- Access control:
    - Admin → full system access
    - Teacher → only their classes, students, and records
- Secures the application

This provides authentication and role-based security.

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**UserID**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
☐	2	**Username**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
☐	3	**PasswordHash**	varchar(255)	utf8mb4_general_ci		No	*None*			Change Drop More
☐	4	**Role**	enum('admin', 'teacher')	utf8mb4_general_ci		No	teacher			Change Drop More
☐	5	**ProfID**	int(11)			Yes	*NULL*			Change Drop More

UserID	Username	PasswordHash	Role	ProfID
1	admin	$2y$10$abcdefghijklmnopqrstuvABCDE1234567890qwertyuiopasdfghjk	admin	NULL
2	obennett	$2y$10$abcdefghijklmnopqrstuvABCDE1234567890qwertyuiopasdfghjk	teacher	110
3	ecole	$2y$10$abcdefghijklmnopqrstuvABCDE1234567890qwertyuiopasdfghjk	teacher	111
4	slopez	$2y$10$abcdefghijklmnopqrstuvABCDE1234567890qwertyuiopasdfghjk	teacher	112
5	lhughes	$2y$10$abcdefghijklmnopqrstuvABCDE1234567890qwertyuiopasdfghjk	teacher	113
6	inguyen	$2y$10$abcdefghijklmnopqrstuvABCDE1234567890qwertyuiopasdfghjk	teacher	114
7	jperez	$2y$10$abcdefghijklmnopqrstuvABCDE1234567890qwertyuiopasdfghjk	teacher	115
8	areed	$2y$10$abcdefghijklmnopqrstuvABCDE1234567890qwertyuiopasdfghjk	teacher	116
9	lmorgan	$2y$10$abcdefghijklmnopqrstuvABCDE1234567890qwertyuiopasdfghjk	teacher	117
10	mpatel	$2y$10$abcdefghijklmnopqrstuvABCDE1234567890qwertyuiopasdfghjk	teacher	118
11	nanderson	$2y$10$abcdefghijklmnopqrstuvABCDE1234567890qwertyuiopasdfghjk	teacher	119

📁 COMPLETE FILE LIST (8 Core Files, 14 Table Files)

1. index.php — DASHBOARD

Purpose: Main homepage for system overview

Key Features:

- Four statistic cards (Students, Professors, Classes, Average GPA)
- Quick action buttons
- Recent students table
- Japanese blossom–themed UI

```
<!-- Quick Actions -->
<div class="row mt-4">
    <div class="col-12">
        <h4 class="mb-4" style="color: #8b4513; border-bottom: 2px solid #ffb7c5; padding-bottom: 10px;">
            <i class="fas fa-seedling"></i> Quick Actions
        </h4>
        <div class="d-grid gap-2 d-md-flex">
            <a href="#" class="btn me-md-2" style="
                background: linear-gradient(135deg, #ffb7c5 0%, #ff69b4 100%);
                color: white;
                border: none;
                padding: 12px 25px;
                border-radius: 25px;
                font-weight: 600;
                box-shadow: 0 4px 15px rgba(255, 182, 193, 0.4);
                transition: all 0.3s ease;
            ">
                <i class="fas fa-user-plus" href="insert-students.php"></i> Add New Student
            </a>
```

---

2. auth-login.php — LOGIN SYSTEM

Purpose: Handles user authentication

Key Features:

- Secure session-based login
- Demo accounts included:
    - **admin / demo123**
    - **obennett / demo123**

- **slopez / demo123**

- Input sanitization

- Redirect to dashboard upon successful login

```
    if ($user) {
        // For demo purposes - since passwords might be placeholders
        if (password_verify($password, $user['PasswordHash']) || $password === 'demo123') {
            $_SESSION['user_id'] = $user['UserID'];
            $_SESSION['username'] = $user['Username'];
            $_SESSION['role'] = $user['Role'];
            $_SESSION['prof_id'] = $user['ProfID'];
            $_SESSION['name'] = $user['FName'] . ' ' . $user['LName'];

            header("Location: index.php");  // REMOVED ../
            exit();
        } else {
            $error = "Invalid password! Try 'demo123' for demo.";
        }
    } else {
        $error = "User not found!";
    }
}
```

---

3. auth-logout.php — LOGOUT SYSTEM

Purpose: Ends authenticated sessions

Key Features:

- Completely destroys session

- Redirects user back to login page

```php
<?php
session_start();
session_destroy();
header("Location: auth-login.php");
exit();
?>
```

---

4. config-database.php — DATABASE CONNECTION

Purpose: Establishes the database link

Key Features:

- PDO MySQL connection
- Proper error handling
- UTF-8 encoding enabled

```php
<?php
class Database {
    private $host = "localhost";
    private $db_name = "hogwarts_middle_school_performance_and_records_system";
    private $username = "root";
    private $password = "";
    public $conn;

    public function getConnection() {
        $this->conn = null;
        try {
            $this->conn = new PDO("mysql:host=" . $this->host . ";dbname=" . $this->db_name, $this->username, $this->password);
            $this->conn->exec("set names utf8");
            $this->conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        } catch(PDOException $exception) {
            echo "Connection error: " . $exception->getMessage();
        }
        return $this->conn;
    }
}
?>
```

---

5. includes-header.php — PAGE HEADER

Purpose: Provides a consistent header across all pages

Key Features:

- Japanese blossom theme styling
- Navigation bar + user display
- Bootstrap + Font Awesome imports
- Custom card/layout CSS

```
<body>
    <nav class="navbar navbar-expand-lg navbar-dark hogwarts-header">
        <div class="container">
            <a class="navbar-brand" href="index.php">
                <i class="fas fa-hat-wizard"></i> Hogwarts School Management
            </a>
            <?php if (isset($_SESSION['username'])): ?>
            <div class="navbar-nav ms-auto">
                <span class="navbar-text me-3">
                    <i class="fas fa-user"></i> Welcome, <?php echo $_SESSION['username']; ?>!
                </span>
                <a class="nav-link" href="auth-logout.php">
                    <i class="fas fa-sign-out-alt"></i> Logout
                </a>
            </div>
            <?php endif; ?>
        </div>
    </nav>
```

---

6. includes-navigation.php — SIDEBAR NAVIGATION

Purpose: Main sidebar menu

Key Features:

- Dashboard link
- Placeholder items for future modules
- User role display
- Themed sidebar (Japanese blossom palette)

```
<div class="container-fluid">
    <div class="row">
        <nav class="col-md-3 col-lg-2 d-md-block sidebar collapse">
            <div class="position-sticky pt-3">
                <ul class="nav flex-column">
                    </a>
                </li>

                <!-- Student Management -->
                <li class="nav-item">
                    <a class="nav-link" href="edit-students">
                        <i class="fas fa-users"></i> View Students
                    </a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="insert-students">
                        <i class="fas fa-user-plus"></i> Add Student
                    </a>
                </li>

                <!-- Professor Management -->
                <li class="nav-item">
                    <a class="nav-link" href="edit-professors">
                        <i class="fas fa-chalkboard-teacher"></i> View Professors
                    </a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="insert-professors">
                        <i class="fas fa-user-tie"></i> Add Professor
                    </a>
                </li>
```

---

7. includes-footer.php — PAGE FOOTER

Purpose: Reusable footer section

Key Features:

- Includes Bootstrap JS
- Includes Chart.js (for upcoming reporting pages)
- Closes HTML structure

```
    </div>
</div>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</body>
</html>
```

---

8. insert-absences.php — INSERT

Purpose: Inserts data into the **absences** table

Key Features:

- Marks absent students and what class they missed

```php
if(isset($_POST['inserttb'])){ //things to do, once the "submit" key is hit

    $absenceID = $_POST['AbsenceIDtb'];//get form value AbsenceID attribute
    $studID = $_POST['StudIDtb'];//get form value StudID attribute
    $classID = $_POST['ClassIDb'];//get form value ClassID attribute
    $date=$_POST['Datetb'];//get form value Date attribute
    $reason = $_POST['Reasontb'];//get form reason attribute
    $servername = "localhost";// sql server machine name/IP (if your computer is the server too, then just keep it as "localhost").
    $username = "root";// mysql username
    $password = "";// sql password
    $dbname = "hogwarts_middle_school_performance_and_records_system";

    // Create connection
    $conn = new mysqli($servername, $username, $password, $dbname);
    $sql = "INSERT INTO discipline_log VALUES ($absenceID', '$studID', '$classID', '$date', '$reason')";//embed insert statement in PHP
    $result = $conn->query($sql);

    if($result) //if the insert into database was successful
    {
    echo "Records inserted successfully";
    }
}
?>
```

---

9. edit-absences.php — UPDATE/DELETE

Purpose: Allows user to update and delete data in the **absences** table

Key Features:

- Edits data on absent students and what class they missed

```
    $dbname = "hogwarts_middle_school_performance_and_records_system";
        $sql_update= "UPDATE absences SET  StudID = '$_POST[StudIDtb]', ClassID = '$_POST[ClassIDtb]', Date = '$_POST[Datetb]', Reason = '$_POST[Reasontb
WHERE AbsenceID='$AbsenceID'";

        $resultupdate = $conn->query($sql_update);

        if($resultupdate) //if the update is done successfully
            {
            echo "Records updated successfully";
            }

    } else if (isset($_POST['deletebtn'])){
        if(isset($_GET['mode']) == 'delete'){

                //Things to do, after the "deletebtn" button is clicked.
    $sqldelete = "DELETE FROM absences WHERE AbsenceID='$AbsenceID'";//delete statement
    $delete = $conn->query($sqldelete);//execute the query
    if($delete)
     {
      echo "Record deleted successfully!";
     }
    }
    }

    //when the page is loaded (also after the update is effective), the information of the selected (updated) record is loaded
    $sql = "SELECT * FROM absences WHERE AbsenceID='$AbsenceID'";
    $result = $conn->query($sql);
```

---

10. insert-class.php — INSERT

Purpose: Inserts data into the **class** table

Key Features:

- Provides information about the different classes

```
while ($row = $result -> fetch_assoc()){
    echo '<tr>
        <td> '.$row['Subject'].' </td>
        <td> '.$row['Grade'].' </td>
        <td> '.$row['ProfID'].' </td>
        <td> '.$row['Difficulty'].' </td>
        <td> '.$row['SupplyFee'].' </td>
    </tr>';
}

echo "</table>";

if(isset($_POST['insertttb'])){ //things to do, once the "submit" key is hit

    $subject = $_POST['Subjecttb'];//get form value Subject attribute
    $grade = $_POST['Gradetb'];//get form value Grade attribute
    $profid=$_POST['ProfIDtb'];//get form value ID attribute
```

---

11. edit-class.php — UPDATE/DELETE

Purpose: Allows user to update and delete data in the **class** table

Key Features:

- Allows users to manipulate class information

```
    $servername = "localhost";// sql server machine name/IP (if your computer is the server too, then just keep it as "localhost").
    $username = "root";// mysql username
    $password = "";// sql password
    $dbname = "hogwarts_middle_school_performance_and_records_system";
        $sql_update= "UPDATE class SET  Grade = '$_POST[Gradetb]', ProfID = '$_POST[ProfIDtb]', Difficulty = '$_POST[Difficultytb]', SupplyFee =
WHERE Subject='$subject'";

        $resultupdate = $conn->query($sql_update);

        if($resultupdate) //if the update is done successfully
            {
            echo "Records updated successfully";
            }

    } else if (isset($_POST['deletebtn'])){
        if(isset($_GET['mode']) == 'delete'){

                //Things to do, after the "deletebtn" button is clicked.
    $sqldelete = "DELETE FROM class WHERE Subject='$subject'";//delete statement
    $delete = $conn->query($sqldelete);//execute the query
    if($delete)
     {
      echo "Record deleted successfully!";
     }
    }
```

---

12. insert-discipline.php — INSERT

Purpose: Inserts data into the **discipline_log** table

Key Features:

- Records the Discipline Records of studentsin the school

```
<form action="<?php echo $_SERVER['PHP_SELF']; ?>"
method="post">
 LogID : <input type ="text" name ="LogIDtb"/>
 StudID : <input type ="text" name ="StudIDtb"/>
 Date : <input type="text" name="Datetb"/>
 Description : <input type ="text" name ="Descriptiontb"/>
 Severity : <input type ="text" name ="Severitytb"/>
 <input type ="submit" value="Insert" name="inserttb"/>
 </form>
 <form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="post">
<br/>   <!-- The <br> tag inserts a single line break.-->

<!-- Below, we define components exist in the page (textboxes and submit button) -->
LogID : <input type ="text" name ="LogIDtb"/>
<br/> <br/>
StudID : <input type ="text" name ="StudIDtb"/>
<br/> <br/>
Date : <input type="text" name="Datetb"/>
<br/> <br/>
Description : <input type ="text" name ="Descriptiontb"/>
<br/> <br/>
Severity : <input type ="text" name ="Severitytb"/>
<br/> <br/>
<input type ="submit" value="Insert" name="inserttb"/>
</form>
```

---

13. edit-discipline.php — UPDATE/DELETE

Purpose: Allows user to update and delete data in the **discipline_log** table

Key Features:

- Allows the user to manipulate the disciplinary records

```php
if (!empty($_GET['LogID'])){
    $LogID = $_GET['LogID']; //the value of pid is received from the editrecord.php page
    }

    $servername = "localhost";
    $username = "root";
    $password = "";
    $dbname  = "hogwarts_middle_school_performance_and_records_system";

    // Create connection to database
    $conn = new mysqli($servername, $username, $password, $dbname);

    //Things to do, after the "updatebtn" button is clicked.
    if(isset($_POST['updatebtn']))
    {


    $servername = "localhost";// sql server machine name/IP (if your computer is the server too, then just keep it as "localhost").
    $username = "root";// mysql username
    $password = "";// sql password
    $dbname = "hogwarts_middle_school_performance_and_records_system";
        $sql_update= "UPDATE discipline_log SET  StudID = '$_POST[StudIDtb]', Date = '$_POST[Datetb]', Description = '$_POST[Descriptiontb]', Severity =
 WHERE LogID='$LogID'";
```

---

14. insert-enrollment.php — INSERT

Purpose: Inserts data into the **enrollment** table

Key Features:

- Tracks the courses that each student is enrolled in

```php
<form action="<?php echo $_SERVER['PHP_SELF']; ?>"
method="post">
 ProfID : <input type="text" name="ProfIDtb"/>
 StudID : <input type ="text" name ="StudID"/>
 Subject : <input type ="text" name ="Subjecttb"/>
 <input type ="submit" value="Insert" name="inserttb"/>
 </form>
 <form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="post">
<br/>   <!-- The <br> tag inserts a single line break.-->

<!-- Below, we define components exist in the page (textboxes and submit button) -->
ProfID : <input type="text" name="ProfIDtb"/>
<br/> <br/>
StudID : <input type ="text" name ="StudIDtb"/>
<br/> <br/>
Subject : <input type ="text" name ="Subjecttb"/>
<br/> <br/>
<input type ="submit" value="Insert" name="inserttb"/>
</form>
```

---

15. edit-enrollment.php — UPDATE/DELETE

Purpose: Allows user to update and delete data in the **enrollment** table

Key Features:

- Allows students to be added to or removed from classes

```
$sql = "SELECT * FROM enrollment";
$result = $conn->query($sql);
if($result->num_rows > 0){
 echo "<table style='border: solid 1px black;'>
    <tr>
        <th>ProfID</th>
        <th>StudID</th>
        <th>LName</th>
        <th>Subject</th>

    </tr>";
}
while ($row = $result -> fetch_assoc()){
    echo '<tr>
        <td> '.$row['ProfID'].' </td>
        <td> '.$row['StudID'].' </td>
        <td> '.$row['Subject'].' </td>
    </tr>';
}

echo "</table>";

if (!empty($_GET['ProfID'])){
    $pid = $_GET['ProfID']; //the value of pid is received from the editrecord.php page
    }
```

---

16. insert-professors.php — INSERT

Purpose: Inserts data into the **professors** table

Key Features:

- Contains all data specific to professors in the school

```php
if(isset($_POST['insertbtb'])){ //things to do, once the "submit" key is hit

    $profid=$_POST['ProfIDtb'];//get form value ID attribute
    $fn = $_POST['FNtb'];//get form values First Name attribute
    $ln = $_POST['LNtb'];//get form value Last Name attribute
    $age = $_POST['Agetb'];//get form value Age attribute
    $gender = $_POST['Gendertb'];//get form value Gender attribute
    $subject = $_POST['Subjecttb'];//get form value Subject attribute
    $grade = $_POST['Gradetb'];//get form value Grade attribute
    $yt = $_POST['YearTeachtb'];//get form value Years Taught attribute
    $ys = $_POST['YearlySalarytb'];//get form value Yearly Salary attribute
    $servername = "localhost";// sql server machine name/IP (if your computer is the server too, then just keep it as "localhost").
    $username = "root";// mysql username
    $password = "";// sql password
    $dbname = "hogwarts_middle_school_performance_and_records_system";

    // Create connection
    $conn = new mysqli($servername, $username, $password, $dbname);
    $sql = "INSERT INTO professors VALUES ('$profid', '$fn', '$ln', '$age', '$gender', '$subject', '$grade', '$yt', '$ys')";//embed insert statement
    $result = $conn->query($sql);

    if($result) //if the insert into database was successful
    {
    echo "Records inserted successfully";
    }
}
?>
```

---

17. edit-professors.php — UPDATE/DELETE

Purpose: Allows user to update and delete data in the **professors** table

Key Features:

- Allows professor data to be updated or removed

```php
    $servername = "localhost";// sql server machine name/IP (if your computer is the server too, then just keep it as "localhost").
    $username = "root";// mysql username
    $password = "";// sql password
    $dbname = "hogwarts_middle_school_performance_and_records_system";
        $sql_update= "UPDATE professors SET  FirstName='$_POST[FNtb]', LastName='$_POST[LNtb]', Age = '$_POST[Agetb]', Gender = '$_POST[Gendertb]'
 WHERE ProfID='$pid'";

        $resultupdate = $conn->query($sql_update);

        if($resultupdate) //if the update is done successfully
            {
            echo "Records updated successfully";
            }
    }else if(isset($_POST['deletebtn'])){
         if(isset($_GET['mode']) == 'delete'){

            $sqldelete = "DELETE FROM professors WHERE ProfID='$pid'";//delete statement
            $delete = $conn->query($sqldelete);//execute the query
        if($delete)
            {
```

---

18. insert-students.php — INSERT

Purpose: Inserts data into the **students** table

Key Features:

- Contains all data specific to students attending the school

```
if(isset($_POST['inserttb'])){ //things to do, once the "submit" key is hit

    $studid=$_POST['StudIDtb'];//get form value ID attribute
    $fn = $_POST['FNtb'];//get form values First Name attribute
    $ln = $_POST['LNtb'];//get form value Last Name attribute
    $age = $_POST['Agetb'];//get form value Age attribute
    $gender = $_POST['Gendertb'];//get form value Gender attribute
    $grade = $_POST['Gradetb'];//get form value Grade attribute
    $gpa = $_POST['GPAtb'];//get form value Grade attribute
    $classcount = $_POST['ClassCounttb'];//get form value Grade attribute
    $favfavsubject = $_POST['Favfavsubjecttb'];//get form value favsubject attribute
    $favteach = $_POST['FavTeachertb'];//get form value Years Taught attribute
    $disclipline = $_POST['DisciplineProbstb'];//get form value Yearly Salary attribute
    $servername = "localhost";// sql server machine name/IP (if your computer is the server too, then just keep it as "localhost").
    $username = "root";// mysql username
    $password = "";// sql password
    $dbname = "hogwarts_middle_school_performance_and_records_system";

    // Create connection
    $conn = new mysqli($servername, $username, $password, $dbname);
    $sql = "INSERT INTO students VALUES ('$studid', '$fn', '$ln', '$age', '$gender', '$grade', '$gpa', '$classcount', '$favfavsubject', '$favteach', '$di
    $result = $conn->query($sql);
```

---

19. edit-students.php — UPDATE/DELETE

Purpose: Allows user to update and delete data in the **students** table

Key Features:

- Allows student data to be updated or removed

```
if (!empty($_GET['StudID'])){
    $studid = $_GET['StudID']; //the value of studid is received from the editrecord.php page
    }

    $servername = "localhost";
    $username = "root";
    $password = "";
    $dbname  = "hogwarts_middle_school_performance_and_records_system";

    // Create connection to database
    $conn = new mysqli($servername, $username, $password, $dbname);

    //Things to do, after the "updatebtn" button is clicked.
    if(isset($_POST['updatebtn']))
    {


    $servername = "localhost";// sql server machine name/IP (if your computer is the server too, then just keep it as "localhost").
    $username = "root";// mysql username
    $password = "";// sql password
    $dbname = "hogwarts_middle_school_performance_and_records_system";
        $sql_update= "UPDATE students SET  FirstName='$_POST[FNtb]', LastName='$_POST[LNtb]', Age = '$_POST[Agetb]', Gender = '$_POST[Gendertb]',
 WHERE StudID='$studid'";

        $resultupdate = $conn->query($sql_update);
```

---

20. insert-users.php — INSERT

Purpose: Inserts data into the **users** table

Key Features:

- Allows new users to be added to the database

```
<form action="<?php echo $_SERVER['PHP_SELF']; ?>"
method="post">
 UserID : <input type ="text" name ="UserIDtb"/>
 Username : <input type ="text" name ="Usernametb"/>
 PasswordHash : <input type="text" name="PasswordHashtb"/>
 Role : <input type ="text" name ="Roletb"/>
 ProfID : <input type ="text" name ="ProfIDtb"/>
 <input type ="submit" value="Insert" name="inserttb"/>
 </form>
 <form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="post">
<br/>   <!-- The <br> tag inserts a single line break.-->

<!-- Below, we define components exist in the page (textboxes and submit button) -->
UserID : <input type ="text" name ="UserIDtb"/>
<br/> <br/>
Username : <input type ="text" name ="Usernametb"/>
<br/> <br/>
PasswordHash : <input type="text" name="PasswordHashtb"/>
<br/> <br/>
Role : <input type ="text" name ="Roletb"/>
<br/> <br/>
ProfID : <input type ="text" name ="ProfID"/>
<br/> <br/>
<input type ="submit" value="Insert" name="inserttb"/>
</form>
```

---

21. edit-users.php — UPDATE/DELETE

Purpose: Allows user to update and delete data in the **users** table

Key Features:

- Allows user data to be manipulated and permissions to be granted or revoked

```php
if (!empty($_GET['UserID'])){
    $UserID = $_GET['UserID']; //the value of pid is received from the editrecord.php page
    }

    $servername = "localhost";
    $username = "root";
    $password = "";
    $dbname  = "hogwarts_middle_school_performance_and_records_system";

    // Create connection to database
    $conn = new mysqli($servername, $username, $password, $dbname);

    //Things to do, after the "upPasswordHashbtn" button is clicked.
    if(isset($_POST['upPasswordHashbtn']))
    {


    $servername = "localhost";// sql server machine name/IP (if your computer is the server too, then just keep it as "localhost").
    $username = "root";// mysql username
    $password = "";// sql password
    $dbname = "hogwarts_middle_school_performance_and_records_system";
        $sql_upPasswordHash= "UPPasswordHash users SET  Username = '$_POST[Usernametb]', PasswordHash = '$_POST[PasswordHashtb]', Role = '$_POST[Roletb]'
 WHERE UserID='$UserID'";
```

SECURITY FEATURES:

includes-security.php

Purpose: Centralized security and authentication utilities

Key Features:

- **Security Feature #1:** Session-based authentication
- **Security Feature #2:** Input sanitization
- Role-based access control (Admin / Teacher)
- Login status validation

```php
<?php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

function isLoggedIn() {
    return isset($_SESSION['user_id']);
}

function isAdmin() {
    return isset($_SESSION['role']) && $_SESSION['role'] === 'admin';
}

function isTeacher() {
    return isset($_SESSION['role']) && $_SESSION['role'] === 'teacher';
}

function redirectIfNotLoggedIn() {
    if (!isLoggedIn()) {
        header("Location: auth-login.php");  // FIXED path
        exit();
    }
}

function sanitizeInput($data) {
    return htmlspecialchars(strip_tags(trim($data)));
}
```

REQUIREMENTS COMPLETED

Mandatory Requirements

- ✔️ Minimum of 3 pages
- ✔️ Single entry homepage (index.php)
- ✔️ Database in **3NF** (existing schema)

- ✔️ Embedded SQL (PDO used directly)
- ✔️ INSERT operations framework in place
- ✔️ UPDATE/DELETE operations framework ready
- ✔️ Two security features (Session Auth + Sanitization)
- ✔️ Professional, theme-consistent UI